Chapter 5

Graph Neural Network

Graph Neural Network, or **GNN**, is a deep learning model designed for **graph-structured data**, where data items are connected by relationships.

In a graph:

- **Nodes (vertices)** represent entities
- **Edges** represent connections or relations between entities

For example:

- In a social network, a person is a node and friendship is an edge
- In a citation network, a paper is a node and citation is an edge
- In a knowledge graph, an entity is a node and a relation is an edge

GNN learns by **passing and aggregating information from neighboring nodes**.

This is often called **message passing**.

Graph Neural Network (GNN)

Deep leaming for graph-structured data

Nodes (Vertices)

Represent Entities

Edges

Represent Relationships

Basic idea:

For a node v, its representation is updated using its own features and the features of its neighbors:

$$h_v^{(k+1)} = \sigma\left(W \cdot \text{AGG}\left(\{h_u^{(k)} : u \in \mathcal{N}(v)\} \cup h_v^{(k)}\right)\right)$$

Where:

- $h_v^{(k)}$= feature vector of node v at layer k
- $\mathcal{N}(v)$= neighbors of node v
- AGG= aggregation function such as sum, mean, or max
- W= learnable weight matrix
- σ= activation function

After several layers, each node representation contains information from its nearby graph structure.

Popular GNN models:

- **GCN** – Graph Convolutional Network
- **GAT** – Graph Attention Network
- **GraphSAGE**
- **GIN** – Graph Isomorphism Network

Common tasks of GNN:

- Node classification
- Link prediction
- Graph classification
- Recommendation systems
- Fraud detection
- Molecule property prediction

Components of a GNN model

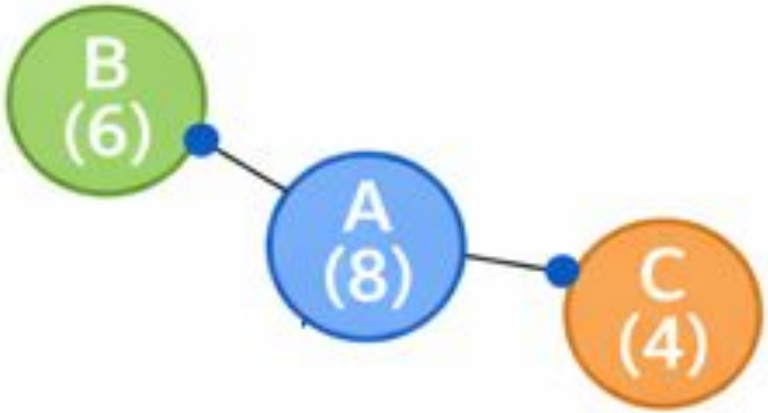
1.Input graph
Contains nodes, edges, and node features

2.Aggregation step
Collects information from neighboring nodes

3.Update step
Combines neighbor information with the current node information

4. Output layer
Produces final prediction or node embedding

For Example



Step	Explanation	Result
Input Graph	**Nodes** A, B, C with features 8, 6, 4	Graph created
Aggregation	Collect neighbor values of A from B and C	((6+4)/2 =5)
Update	Combine A's own value with neighbor value	(8+5=13)
Output	Predict class using updated value	High Performer

Neural Message Passing

Neural Message Passing means that each node in a graph receives information from its neighboring nodes, combines it with its own information, and then updates its representation.

Neural Message Passing

Step	Neural Message Passing Process	Mathematical Form	Result
Input Graph	The graph contains three nodes A, B, and C with feature values 8, 6, and 4	$h_A = 8,\ h_B = 6,\ h_C = 4$	Initial node features are assigned
Message Passing	Neighbor nodes B and C send their information to node A	$m_{B \to A} = 6,\ m_{C \to A} = 4$	A receives messages from its neighbors
Aggregation	Node A combines the received messages from B and C using mean operation	$m_A = \frac{6+4}{2} = 5$	Aggregated neighbor message for A is 5
Update	Node A updates its value by combining its own feature with aggregated message	$h'_A = h_A + m_A = 8 + 5 = 13$	Updated node representation becomes 13
Output	The updated node value is used for prediction	$13 > 10$	Node A is predicted as **High Performer**

conclusion

Node	Own Value	Neighbor Values	Aggregated Message	Updated Value	Prediction
A	8	6, 4	5	13	High Performer

Generalized Neighborhood Aggregation

Generalized Neighborhood Aggregation is the process in a Graph Neural Network where a node collects information from its neighboring nodes using a more flexible aggregation rule, not only simple mean or sum.

Generalized Neighborhood Aggregation

For a node v, its neighbors are written as $\mathcal{N}(v)$.

The node gathers messages from all its neighbors and combines them.

General form:

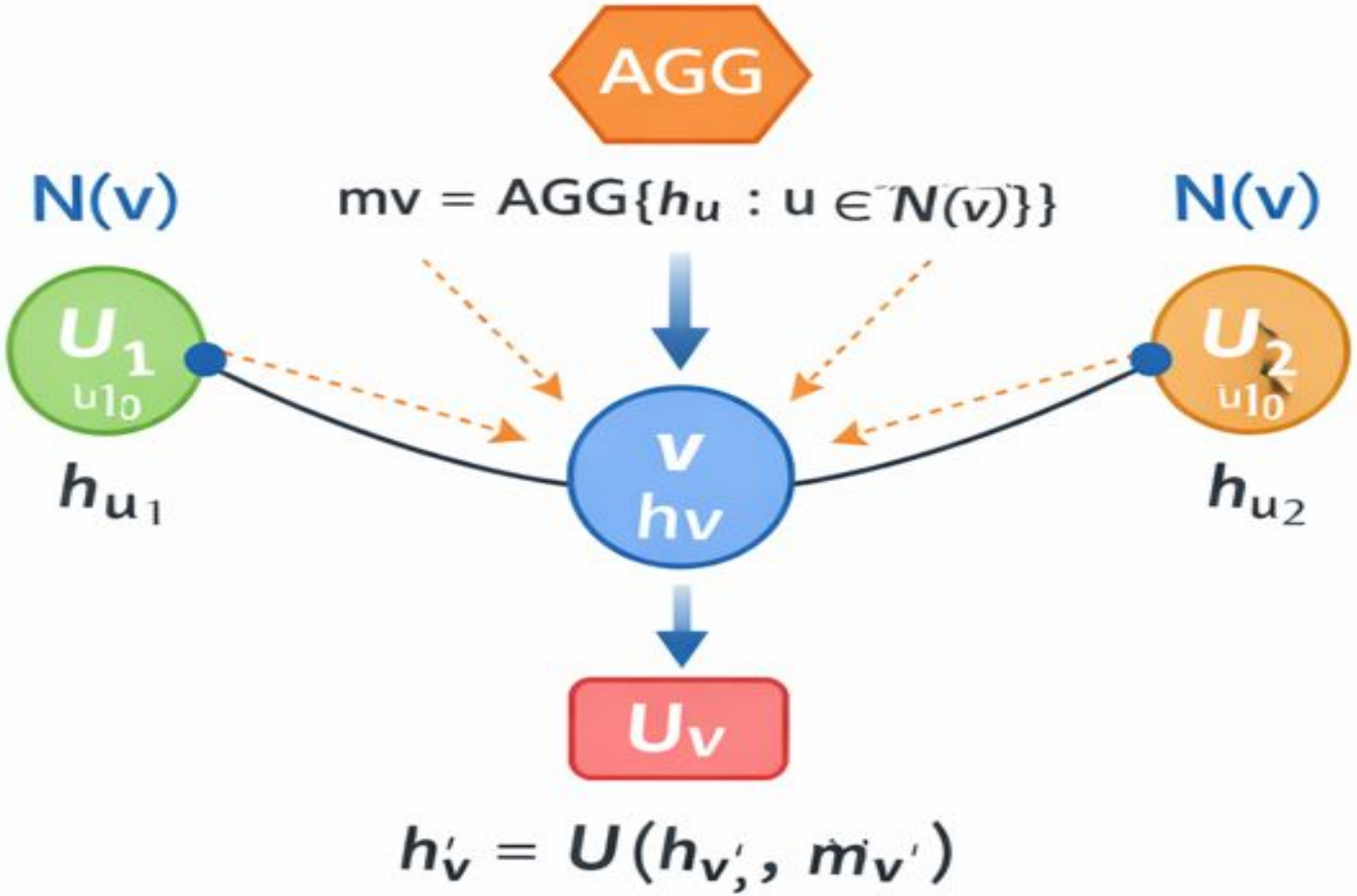
$$m_v = \text{AGG}(\{h_u : u \in \mathcal{N}(v)\})$$

Then the node updates its representation as:

$$h'_v = U(h_v, m_v)$$

Where:

- h_v= current feature of node v
- h_u= feature of neighbor node u
- AGG= aggregation function
- U= update function
- h'_v= updated node feature


AGG
N(v)
mv = AGG{hu : u ∈ N(v)}
N(v)
U1
u1₀
U2
u1₀
v
hv
hu1
hu2
Uv
h'v = U(hv, mv)

It is called **generalized** because the aggregation can be done in many ways, such as:

- **Sum aggregation**

$$m_v = \sum_{u \in \mathcal{N}(v)} h_u$$

- **Mean aggregation**

$$m_v = \frac{1}{|\,\mathcal{N}(v)\,|} \sum_{u \in \mathcal{N}(v)} h_u$$

- **Max aggregation**

$$m_v = \max_{u \in \mathcal{N}(v)} h_u$$

- **Weighted aggregation**

$$m_v = \sum_{u \in \mathcal{N}(v)} \alpha_{vu}\, h_u$$

Here, α_{vu} gives different importance to different neighbors.

Aggregation Methods

Aggregation Type	Formula	Result for neighbors 6 and 4
Sum	$6 + 4$	10
Mean	$(6 + 4)/2$	5
Max	$\max(6,4)$	6

Generalized Update Methods

- **Generalized Update Methods** in Graph Neural Networks are the different ways a node updates its feature after receiving and aggregating messages from its neighbors.
- After aggregation, node v gets a message m_v.

  Then it updates its representation using an update function U:

$$h'_v = U(h_v, m_v)$$

Where:

- h_v =current feature of node v
- m_v =aggregated message from neighbors
- U= update function
- h'_v =updated node feature

Update Methods

Update Method	Formula	Meaning
Sum update	$h'_v = h_v + m_v$	Directly adds self and neighbor information
Weighted update	$h'_v = W_1 h_v + W_2 m_v$	Gives different importance to both parts
Nonlinear update	$h'_v = \sigma(W_1 h_v + W_2 m_v)$	Adds activation for better learning
Concatenation update	$h'_v = W[h_v \parallel m_v]$	Joins both values and transforms them

update results

Update Method	Formula	Calculation	Updated Value
Sum update	$h'_v = h_v + m_v$	$8 + 5$	13
Weighted update	$h'_v = 0.6h_v + 0.4m_v$	$0.6(8) + 0.4(5)$	6.8
Nonlinear update	$h'_v = \text{ReLU}(0.6h_v + 0.4m_v)$	$\text{ReLU}(0.6(8) + 0.4(5))$	6.8
Concatenation update	$h'_v = W[h_v \parallel m_v]$	$W[8 \parallel 5]$	depends on W
GRU/LSTM update	$h'_v = \text{GRU}(h_v, m_v)$	GRU(8,5)	learned value

Graph Pooling

- **Graph Pooling** is a technique in Graph Neural Networks used to **reduce the size of a graph while preserving its important information**.
- It is similar to pooling in CNNs, where the size of an image feature map is reduced.

  In GNNs, graph pooling reduces:
  - number of **nodes**
  - number of **edges**
  - graph complexity
- so that the model can learn a compact and meaningful graph representation.

A graph may contain many nodes, but not all nodes are equally important.

Graph pooling selects or combines important nodes and creates a smaller graph.

If the original graph is:

$$G = (V, E)$$

after pooling, it becomes a smaller graph:

$$G' = (V', E')$$

where:

$$| V' | < | V |$$

That means the pooled graph has fewer nodes than the original graph.

Why graph pooling is needed

- reducing computation
- removing less important nodes
- capturing global graph structure
- improving graph classification tasks

General form

If node feature matrix is H, pooling transforms it into a reduced feature matrix H':

$$H' = \text{Pool}(H)$$

or more generally:

$$(H', A') = \text{Pool}(H, A)$$

Where:

- H= original node features
- A= adjacency matrix
- H'= pooled node features

A'= pooled adjacency matrix

Types of graph pooling

1. Global Pooling

This combines all node features into one graph-level vector.

Common methods:

- **Global Sum Pooling**

$$h_G = \sum_{v \in V} h_v$$

- **Global Mean Pooling**

$$h_G = \frac{1}{|V|} \sum_{v \in V} h_v$$

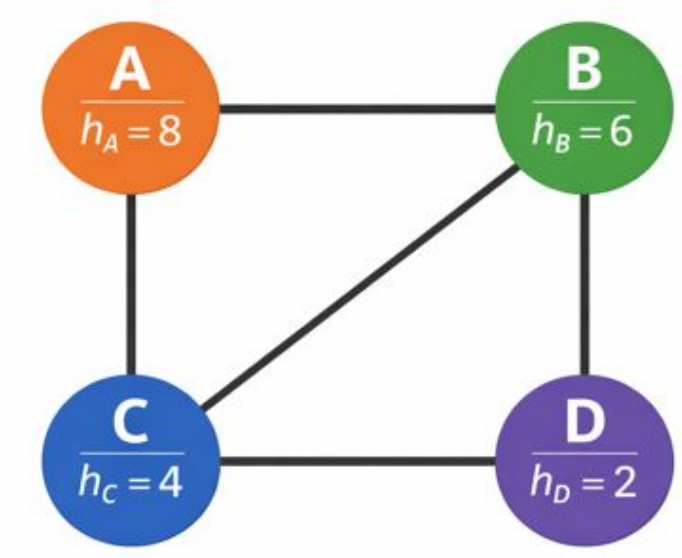
- **Global Max Pooling**

$$h_G = \max_{v \in V} h_v$$

Simple example

Suppose a graph has 4 nodes with feature values:

$$h_A = 8, h_B = 6, h_C = 4, h_D = 2$$



Global Sum Pooling

$$h_G = 8 + 6 + 4 + 2 = 20$$

Global Mean Pooling

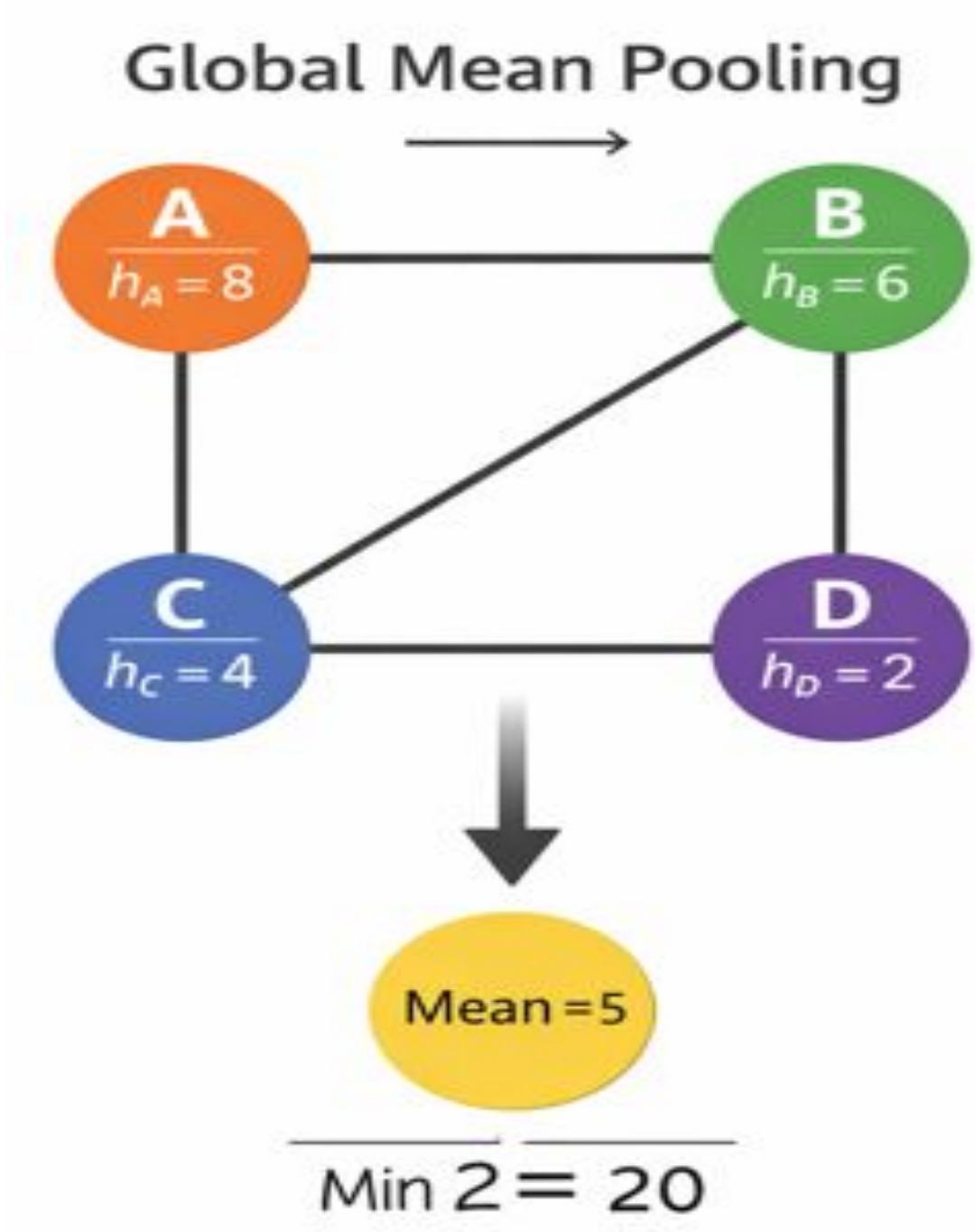
$$h_G = \frac{8 + 6 + 4 + 2}{4} = 5$$

Global Max Pooling

$$h_G = \max(8,6,4,2) = 8$$

In the Table

Pooling Type	Formula	Result
Global Sum Pooling	$8 + 6 + 4 + 2$	20
Global Mean Pooling	$(8 + 6 + 4 + 2)/4$	5
Global Max Pooling	$\max(8,6,4,2)$	8

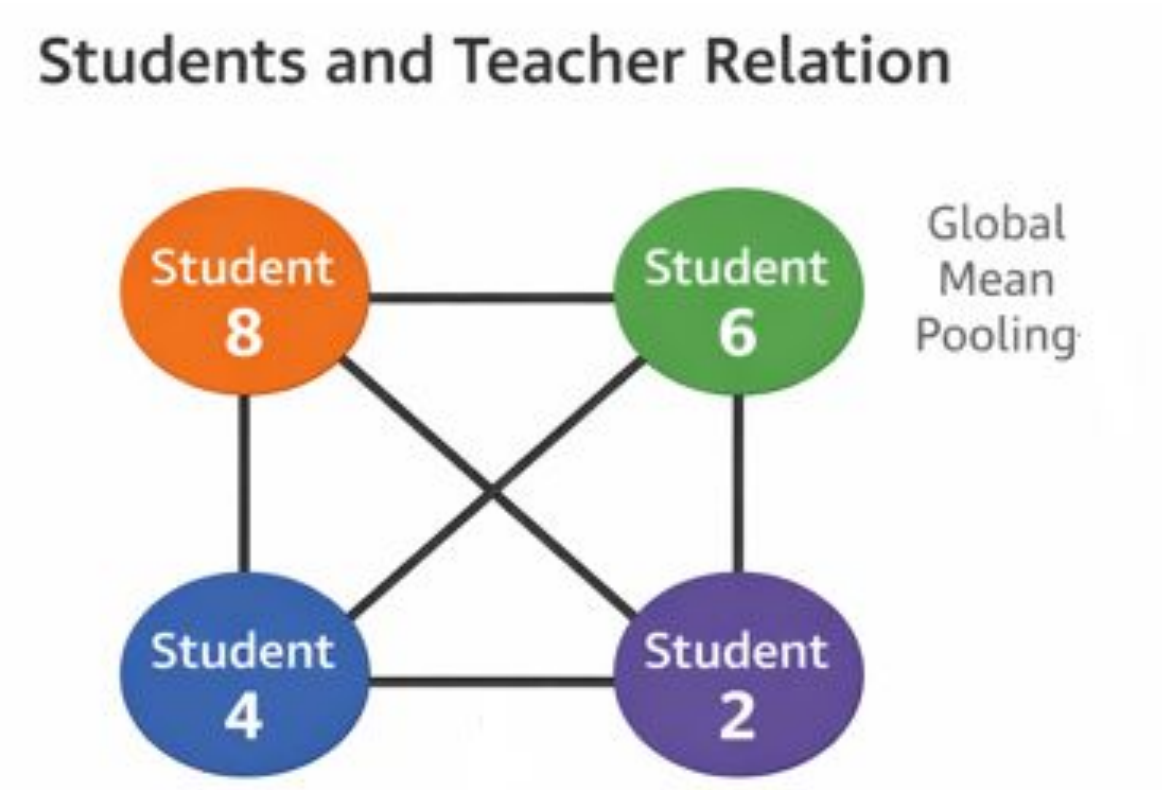

Global Mean Pooling
A
$h_A = 8$
B
$h_B = 6$
C
$h_C = 4$
D
$h_D = 2$
Mean = 5
Min 2 = 20

This graph explains **Global Mean Pooling** using a **students–teacher relation** example.

At the top, there are **4 student nodes**

with feature values:

- Student 1 = **8**
- Student 2 = **6**
- Student 3 = **4**
- Student 4 = **2**



These values may represent marks, performance, activity score, or any node feature.

So the teacher node at the bottom is not another normal student node. It represents the **pooled graph-level output**.

The mean is computed as:

$$\text{Mean} = \frac{8 + 6 + 4 + 2}{4} = \frac{20}{4} = 5$$

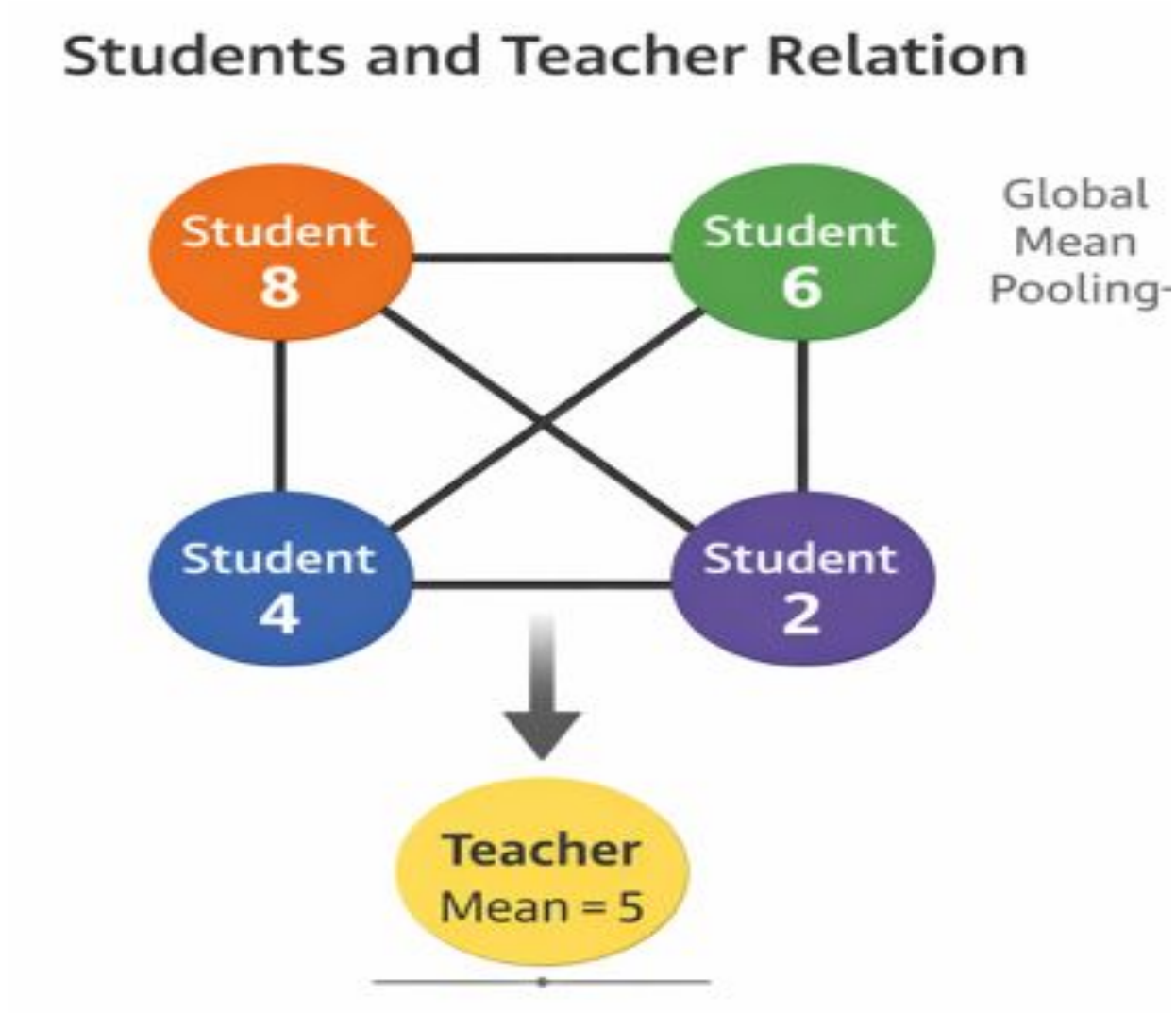
So after pooling, the whole student graph is summarized into **one value = 5**.

Meaning of the graph

- **Before pooling**: 4 student nodes
- **After pooling**: 1 graph representation
- **Teacher node**: acts like a final summary node
- **Output**: average student feature = **5**

Interpretation in simple words

The teacher looks at all 4 students together and calculates the **average performance**.

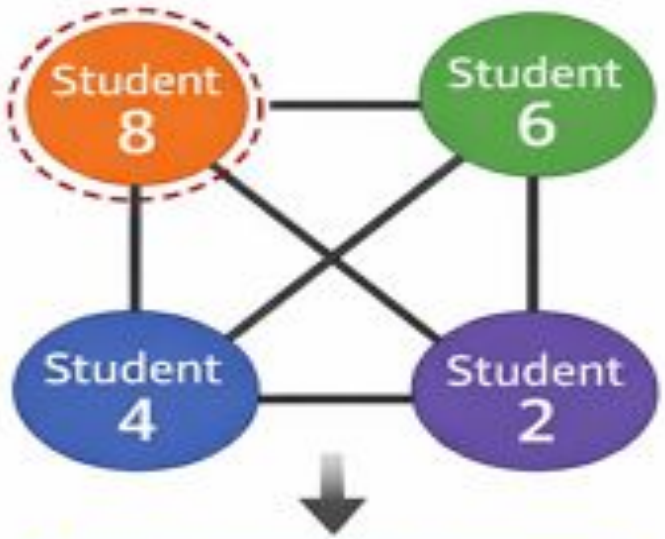

Students and Teacher Relation
Student 8
Student 6
Student 4
Student 2
Global Mean Pooling-
Teacher
Mean = 5

Graph Convolutions

- **Graph convolution** is the core operation used in many **Graph Neural Networks (GNNs)** to update the feature of each node by combining information from its connected neighbors.
- In ordinary convolution on images, a pixel is updated by looking at nearby pixels in a fixed grid.

Graph Convolutions

Graph convolutions update node features by aggregating information from neighboring nodes.



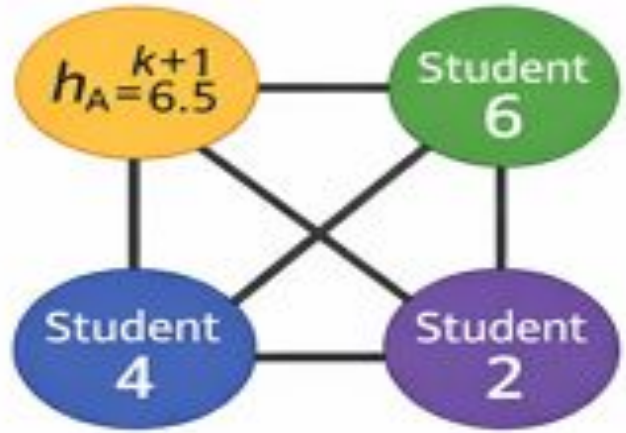
Update Node A

$$h_{A+1} = \sigma(W \cdot AGG(h_A^{k1}))$$

$$m_A = AGG(h_B, h_C, h_A 1)$$

Aggregate from Neighbors

$$6.5 = \sigma(W \cdot AGG(6,4,8)))$$



Updated Node A:

Node A's feature is now updated based on its neighbors

General mathematical form

A general graph convolution rule is:

$$h_v^{(k+1)} = \sigma\left(W \cdot \text{AGG}\left(\{h_u^{(k)} : u \in N(v) \cup \{v\}\}\right)\right)$$

Meaning of each term

- $h_v^{(k)}$: feature of node $v$at layer k
- $N(v)$: set of neighbors of node v
- $\cup \{v\}$: includes the node itself
- AGG: aggregation function
- W: learnable weight matrix
- σ: activation function such as ReLU
- $h_v^{(k+1)}$: updated feature after the next layer